

03/08/2026

K. Sandhiya
(192521378)

8-bit 2's complement Arithmetic

Given:

* Number 1 = -52

* Number 2 = +19

Step 1: convert Decimal to Binary
(+52)

$$52 = 00110100$$

To represent -52 (2's complement):

1. one's complement

$$00110100 \rightarrow 11001011$$

2. Add 1

$$11001011$$

$$+ 00000001$$

$$= 11001100$$

Therefore,

$$-52 = 11001100$$

$$+19 = 00010011$$

(A) Addition: $(-52) + (+19)$

$$\begin{array}{r} 11001100 \\ + 00010011 \\ \hline 11011111 \end{array}$$

Result

11011111 is negative

find decimal value

1. one's complement

$$1101111 \rightarrow 00100000$$

2. Add 1

$$00100001 = 33$$

$$= -33.$$

final answer

$$\text{Decimal} = -33$$

$$\text{Binary} = 1101111$$

$$\text{overflow} = \text{No}$$

B. Subtraction

$$(-52) - (+19)$$

$$-52 + (-19)$$

convert 19 to negative

$$+19$$

$$00010011$$

one's complement

$$11101100$$

Add 1

$$11101101$$

Therefore

$$-19 = 11101101$$

Now add

$$\begin{array}{r} 11001100 \\ + 11101101 \\ \hline 11011001 \end{array}$$

Discard carry

$$1011001$$

Since MSB = 1

Negative number.

Take 2's complement.

One's complement

01000110

Add 1

01000111

= 71

= -71

overflow

Negative + Negative = Negative

No overflow

Processor overflow Detection.

* carry into sign bit

* carry out of sign bit

If different

overflow = Yes

otherwise

overflow = No

Addition $\rightarrow$ No overflow

Subtraction $\rightarrow$ No overflow.

2.

carry Look Ahead Adder (CLA)

Given

Assume

$$A = 10110110$$

$$B = 01100101$$

Generate signal

$$G_i = A_i \neq B_i$$

Eg

$$1 \times 0 = 0$$

$$0 \times 1 = 0$$

$$1 \times 1 = 1$$

Generated only when both bits are 1.

Propagate signal

$$P_i = A_i \oplus B_i$$

Eg:

$$1 \oplus 0 = 1$$

$$0 \oplus 0 = 0$$

$$1 \oplus 1 = 0$$

Carry propagate only when XOR = 1.

Carry Equations

$$C_1 = C_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

All carries are calculated simultaneously.

Ripple carry adder

Each carry waits for previous carry

$$\text{Bit } 0 + \text{Bit } 1 + \text{Bit } 2 + \text{Bit } 3 + \dots$$

Delay increases with number of bits

Carry Look Ahead adder

All carries generated together

C_1

C_2

C_3

C_4

Simultaneously

Time comparison

Ripple carry

$$\text{Delay} = O(n)$$

Carry Look Ahead

$$\text{Delay} = O(\log n)$$

Much faster.

3. Booth's Algorithm

Multiply

$$+11 \times -7$$

Represent using 5 bits

$$+11 = 01011$$

$$-7 = 11001$$

Initial Registers

$$A = 00000$$

$$Q = 11001$$

$$Q_{-1} = 0$$

$$M = 01011$$

$$-M = 10101$$

Iteration-1

$$Q_0 Q_{-1}$$

$$10$$

Subtract M

$$A = A - M$$

Arithmetic Right Shift.

Iteration 2

$$Q_0 Q_{-1}$$

$$11$$

No operation
Shift.

Iteration 3

QOR - 1

01

Add M

Shift

Iteration 4

QOR - 1

10

Subtract M

Shift

Iteration 5

Repeat

After binal shift

110110011

Decimal

-77

binal Answer

$(11 \times (-77))$

$= -77$

Advantages of Booth Algorithm

- * Reduces number of additions
- * Efficient for consecutive 1s
- * Handles signed number directly
- * faster multiplication
- * used in processors.

4. Division (25 ÷ 4)

Dividend = 25

11001

Divisor = 4

00100

A) Restoring Divisor

initial

A = 00000

Q = 11001

Shift Left

Subtract divisor

If negative

Restore remainder

set quotient bit = 0

else

set quotient bit = 1

Repeat 5 times.

final

Quotient = 00110

= 6

Remainder = 00001

= 1

B. Non-Restoring Division

Initial

$$A = 00000$$

$$Q = 11001$$

shift

If remainder positive

Subtract divisor

If negative

Add divisor

No restoring step

final:

$$\text{Quotient} = 00110$$

$$= 6$$

$$\text{Remainder} = 00001$$

$$= 1$$

Comparison

Restoring Division	Non-Restoring Division
Restores remainder	No restoration
More clock cycles	Fewer clock cycles
Slower	Faster
Simple hardware	Slightly complex hardware

Preferred Method.

Non restoring division because

* faster

* less hardware delay

* Better Processor efficiency.

5.

Question: 5

IEEE: 754 floating point

26.75

Binary conversion

Integer

26 = 11010

Fraction

0.75

= .11

Binary

11010.11

Normalize

1.101011×2^4

Single Precision (32-bit)

sign

0

Exponent

$4 + 127$

= 131

Precision Comparison

Single Precision (32-bit):

- * 1 sign bit
- * 8 exponent bits
- * 23 mantissa bits
- * Lower Precision (~ 7 decimal digits).

Double Precision (64-bit)

- * 1 sign bit
- * 11 exponents bits
- * 52 mantissa bits
- * High Precision ($\sim 15-16$ decimal digits).